

VETRA — Admin & Operations Guide

This is the document for whoever **operates** VETRA day to day, rather than builds it. `BACKEND_GUIDE.md` covers the API and the data model, `backend/README.md` covers running and deploying the code, `DOCUMENTATION.md` covers what each page does. This one covers the live services VETRA runs on, how the database is put together and why, how to reach it directly when the admin console itself cannot help (locked out, no Super Admin left, need to look something up by hand), and how to create a Super Admin without ever opening the console.

It is a runbook, not a feature list. Keep it updated whenever a service, credential, or piece of the schema changes in a way that would make a page of this wrong.

1. The services VETRA runs on

Each of these is a separate account on a separate site with its own login. There is no single dashboard showing all of them.

Service	What it does for VETRA	Where you manage it
GitHub	Holds the source (<code>HuddlesTechnologies/VETRA</code>). Pushing to <code>main</code> is what triggers a Render redeploy.	github.com, with an account that has access to the <code>HuddlesTechnologies</code> org.
Render	Runs the backend API (<code>backend/</code>) at <code>https://vetra-api-11an.onrender.com</code> . Free tier.	dashboard.render.com, the <code>vetra-api</code> service.
Vercel	Serves the static frontend (everything outside <code>backend/</code>) at <code>https://vetra-vercel.vercel.app</code> .	vercel.com, the <code>vetra-vercel</code> project.
Clever Cloud	Hosts the MySQL database every piece of real data lives in. Free "Dev" plan.	console.clever-cloud.com, the MySQL add-on.
Cloudinary	Every uploaded file: avatars, cover photos, product images and video, site banners, KYC documents. Nothing is ever written to local disk.	cloudinary.com dashboard.
Resend	Every transactional email: password resets, admin invites, 2FA codes, order updates, KYC decisions, account status changes, low-stock alerts.	resend.com dashboard.
Anthropic	The AI shopping assistant's replies (<code>P0ST /api/assistant/chat</code>).	console.anthropic.com.
Paystack	Bank list and NUBAN-to-account-name resolution for vendor payout accounts. Read-only Miscellaneous API calls only. No charge or transfer integration exists.	dashboard.paystack.com.
CheckID.ng	Identity verification for vendor KYC: NIN, driver's licence, and CAC registration lookups.	checkid.ng, with the sandbox at <code>sandbox.checkid.ng</code> .
Smartsupp	A third-party live-chat widget embedded on the public site and the customer and vendor apps (not the admin console). Support chat with VETRA, not part of the app's own data, and not buyer-to-vendor messaging, which does not exist.	smartsupp.com dashboard.

How a change actually goes live

This trips people up, so it is worth stating plainly.

- **Backend changes** (anything in `backend/`): pushing to `main` on GitHub is enough. Render watches the repo and redeploys, usually within 30 to 60 seconds of the push landing. The deploy runs `npm run migrate && npm start` , so a new migration file applies itself on that same deploy.
- **Frontend changes** (everything else: HTML/CSS/JS at the project root, `customer/` , `vendor/` , `admin/`): pushing to GitHub does **not** update the live site. Vercel here is driven from the command line, not a GitHub integration. You have to run `vercel --prod --yes` from a machine with the Vercel CLI installed and linked to this project (see `.vercel/project.json`) *after* pushing. Forgetting this is the single most common reason for "I pushed a fix but the live site still shows the old version."

Reconnecting or rotating a service

Every integration is reached through an environment variable read at startup. Nothing is hardcoded to a specific account beyond that. To reconnect, replace, or rotate one:

1. Get the new credentials from that service's own dashboard.
2. Render dashboard → `vetra-api` → **Environment** → update the matching variables.

3. Save. Render redeploys automatically on an environment change, same as a code push.

Service	Variables	Notes on rotating
Database	DB_HOST , DB_PORT , DB_NAME , DB_USER , DB_PASSWORD , DB_SSL , DB_CONNECTION_LIMIT , DB_QUEUE_LIMIT	Moving to a different database means running the migrations against it first. Since the Render start command includes <code>npm run migrate</code> , pointing the service at an empty database and redeploying will build the schema, but it will be empty; migrating <i>data</i> is a separate job.
Cloudinary	CLOUDINARY_CLOUD_NAME , CLOUDINARY_API_KEY , CLOUDINARY_API_SECRET	Rotating the key and secret does not touch any file already uploaded. Only new uploads use the new credentials. Changing the cloud name is different: every <code>res.cloudinary.com</code> URL already stored in the database points at the old cloud and would break.
Resend	RESEND_API_KEY , EMAIL_FROM	<code>EMAIL_FROM</code> must be on a domain verified in that Resend account (Domains tab). An unverified domain, or a typo in it, makes every send fail, logged server-side, see §6. Rotating the API key alone does not require re-verifying the domain.
Anthropic	ANTHROPIC_API_KEY , ASSISTANT_MODEL	The assistant errors if the key is invalid or the account is out of credit. Nothing else is affected.
Google Sign-In	GOOGLE_CLIENT_ID	Not secret. It is embedded in the frontend's own JS (<code>google-signin.js</code>), so seeing it in the browser is expected. Changing it means creating a new OAuth Client ID in Google Cloud Console and updating both this variable and <code>google-signin.js</code> , then redeploying the frontend.
JWT signing	JWT_SECRET , JWT_EXPIRES_IN	Rotating <code>JWT_SECRET</code> signs every user out immediately , since every previously issued token fails signature verification on its next request. That is the intended effect if you are rotating because it leaked. Do not touch it casually.
Encryption at rest	ENCRYPTION_KEY	Encrypts vendor bank account numbers and KYC identity numbers (AES-256-GCM). Rotating this without a re-encryption step makes every already-saved payout account and identity number unreadable , since the ciphertext was written under the old key. Every affected vendor would have to re-enter both.
Paystack	PAYSTACK_SECRET_KEY	Read-only calls only, so rotating does not touch any saved payout account: <code>payout_account_number_enc</code> and <code>payout_bank_code</code> are stored independently, not as a live reference. The test key (<code>sk_test...</code>) works identically for both calls this app makes.
CheckID.ng	CHECKID_API_KEY , CHECKID_BASE_URL	Server-side only; this token must never appear in frontend JavaScript. <code>CHECKID_BASE_URL</code> is currently the sandbox (<code>https://sandbox.checkid.ng</code> , set in <code>render.yaml</code>). Pointing it at production is a deliberate switch, not a default.
Session behaviour	SESSION_IDLE_TIMEOUT_MINUTES	Server-side inactivity window, default 30 minutes. Lowering it signs idle people out sooner; raising it is a real security tradeoff, not a convenience setting.
Frontend origin	CORS_ORIGINS , FRONTEND_URL	<code>CORS_ORIGINS</code> is the allow-list of origins permitted to call the API at all. If it does not include wherever the frontend is actually served from, every request fails in the browser with a CORS error rather than a readable API error. Unset falls back to the known Vercel origin, never <code>*</code> . <code>FRONTEND_URL</code> is used only to build links inside emails (the password-reset link) and is not set in <code>render.yaml</code> , so it uses its code default of <code>https://vetra-vercel.vercel.app</code> .

Disconnecting a service

There is no off switch for most of these. Disconnecting means either deleting the variables on Render (the feature degrades in a specific way) or closing the account on that service.

- **Cloudinary removed:** file uploads fail with a real error. Nothing else breaks, but note that existing images keep working only while the Cloudinary account itself still exists.
- **Resend removed:** every email attempt fails silently to the end user. The action it was attached to (invite, reset, order update) still completes, and the code, link, or password is logged to Render's console instead. Grep the logs for `[email:not-configured]` or `[email:failed]` .
- **Anthropic removed:** the assistant chat errors. Nothing else is affected, and nothing currently calls it from the frontend anyway.
- **Paystack removed:** `src/utils/paystack.js` throws "Bank verification isn't configured on this deployment yet", which surfaces to the vendor as a generic 500 rather than a clear reason. Payout account setup stops working; nothing else is affected.

- **CheckID.ng removed:** POST /api/vendors/me/kyc/verify returns 503 "Identity verification is not configured yet." Vendors can still upload documents; nothing can be auto-verified, so every submission would need the manual-review path.
 - **Google Sign-In removed:** "Continue with Google" fails. Email and password auth is unaffected.
 - **The database cannot be disconnected without taking the whole app down.** Every route depends on it.
-

2. How the database is set up, and why

One MySQL database, 19 tables, no ORM. Every query in `backend/src/routes/*.js` is plain SQL through `mysql2/promise`. The schema lives in `backend/migrations/*.sql`, applied in filename order and tracked in `schema_migrations` (itself one of the 19). This section explains reasoning the migration files do not always spell out.

One `users` table for all three account types

Buyers, vendors, and admins are rows in the same table, distinguished by `role` (`buyer` / `vendor` / `admin`). Three tables would mean three copies of the auth, session, and password-reset logic for what is one concept. Role-specific columns sit `NULL` on rows that do not use them.

Current `users` columns:

Column	Type	Notes
<code>id</code>	<code>CHAR(36)</code>	A UUIDv7, not v4. Time-ordered on purpose, see below.
<code>role</code>	<code>ENUM('buyer', 'vendor', 'admin')</code>	Which app this account belongs to.
<code>name</code>	<code>VARCHAR(190)</code>	Display name.
<code>first_name</code> , <code>middle_name</code> , <code>last_name</code>	<code>VARCHAR(100)</code>	Legal name parts, required for vendor signup. Compared field by field against what CheckID.ng returns during KYC. <code>name</code> remains the display field everywhere else.
<code>email</code>	<code>VARCHAR(190)</code>	Unique per role . The same address can exist once as a buyer and once as a vendor, never twice within one role.
<code>phone</code> , <code>address</code>	<code>VARCHAR</code>	Required by the signup routes, nullable on the column.
<code>state</code>	<code>VARCHAR(60)</code>	One of Nigeria's 36 states or the FCT, validated against a fixed list in <code>src/utils/nigerianStates.js</code> . Required for every buyer and vendor signup including Google. Admin rows have none.
<code>password_hash</code>	<code>VARCHAR(255)</code>	bcrypt, one-way . There is no way to look up someone's password. See §4.
<code>status</code>	<code>ENUM('active', 'suspended', 'pending', 'rejected', 'deleted')</code>	<code>pending</code> = vendor awaiting approval. <code>suspended</code> = admin suspension or the account holder's own "Deactivate", same end state either way, reversible. <code>rejected</code> = vendor application denied. <code>deleted</code> = terminal, see below.
<code>signup_method</code>	<code>VARCHAR(40)</code>	<code>email</code> or <code>google</code> .
<code>avatar_url</code> , <code>store_cover_url</code>	<code>VARCHAR(500)</code>	Cloudinary URLs, never a local path or a base64 blob.
<code>store_name</code> , <code>store_category</code> , <code>store_description</code>		Vendor-only.
<code>admin_role</code>	<code>ENUM('Super Admin', 'Moderator', 'Support')</code>	Admin-only. Every <code>requireAdminRole(...)</code> check in the backend gates on this, not on <code>role</code> .
<code>kyc_email_alerts_enabled</code>	<code>BOOLEAN</code>	Per-admin opt-out of the KYC alert email, default true.
<code>two_factor_enabled</code>	<code>BOOLEAN</code>	Email one-time-code sign-in. Only surfaced in the UI for vendors and admins.
<code>payout_bank_name</code> , <code>payout_bank_code</code>	<code>VARCHAR</code>	Vendor-only. <code>payout_bank_code</code> is Paystack's code, needed to re-resolve the account or eventually pay out through Paystack's Transfer API.
<code>payout_account_name</code>	<code>VARCHAR(190)</code>	Vendor-only but not vendor-typed . Resolved from Paystack against the bank code and NUBAN, both for the form's preview and again server-side before it is written. A vendor cannot submit a name the bank does not have on file.
<code>payout_account_number_enc</code>	<code>VARCHAR(255)</code>	The bank account number, AES-256-GCM encrypted (key = <code>ENCRYPTION_KEY</code>). Never stored or returned in plaintext. Every API response shows only a masked <code>.... 1234</code> , derived by decrypting server-side and masking, never by storing a masked copy.

Column	Type	Notes
last_login_at , last_login_ip		The most recent successful sign-in. Full history is in login_ip_history .
last_activity_at	DATETIME	Updated by the auth middleware on every authenticated request. Drives the idle-session timeout.
session_version	INT	Incrementing this invalidates every token already issued for the account.
password_changed_at	DATETIME	Set only by a real password change. NULL means never changed since signup, which the UI states honestly instead of faking a date.
created_at	DATETIME	Real UTC, see the timestamp note below.

What **status = 'deleted'** means. Deleting the row is not possible without breaking every past order, report, and review that legitimately still references that account for someone else's records. Instead the delete flow scrubs personal fields (name → Deleted User , email → a unique deleted-<id>@vetra.deleted placeholder, password_hash → a random unusable value, phone/address/avatar/store/payout fields cleared) and, for a vendor, delists every product, all in one transaction. The row survives for referential integrity, nobody can sign into it, and it shows as "Deleted User" wherever it is still referenced. The placeholder email also frees the original address, so the same person can sign up again later.

Both self-deletion (POST /api/auth/delete-account) and admin deletion (DELETE /api/admin/customers/:id or /vendors/:id , Super Admin only) do the same thing.

Sessions are not just the token

A JWT alone would mean a suspension takes up to seven days to bite. It does not, because src/middleware/auth.js re-reads the account row on every authenticated request and checks four things: the account exists, its status is not suspended or deleted , the token's sessionVersion still matches users.session_version , and last_activity_at is within SESSION_IDLE_TIMEOUT_MINUTES (default 30).

Practical consequence for operators: **suspending an account, changing its role, changing its email, or resetting its password all take effect on that account's very next request**, not whenever their token expires. session_version is bumped by every one of those actions.

Why the IDs look like they do (UUIDv7)

Every primary key is a CHAR(36) UUID generated in backend/src/utls/id.js , but not by crypto.randomUUID() . That produces a UUIDv4, fully random, which InnoDB handles badly as a clustered-index primary key: random insert order means every insert can land anywhere in the B-tree, causing page splits and fragmentation as a table grows. id.js hand-builds a **UUIDv7** instead: the first 48 bits are the current Unix timestamp in milliseconds, the rest random, so new rows insert in roughly chronological order. The on-disk format is identical to a v4, so nothing else had to change.

Why money is stored the way it is

Every price and total (products.price , orders.total , order_items.price_at_purchase) is an INT counted in **kobo** (1 naira = 100 kobo), never a decimal or float naira value. Storing currency as a float invites the classic rounding bug where ₦0.1 + ₦0.2 is not exactly ₦0.3 in binary floating point. An integer count of the smallest unit sidesteps it entirely. Conversion happens only at the UI boundary, in api-client.js's formatNaira() and nairaToKobo() , the only two places in the codebase that multiply or divide by 100. If a raw database number looks 100x too large, that is why.

Timestamps are real UTC

The connection used to set dateStrings: true , which returned every DATETIME as a plain "2026-09-16 21:04:32" string with no timezone marker. A browser in Nigeria (WAT, UTC+1) parses that as *its own* local time, silently shifting every displayed timestamp an hour into the past. It showed up as a fresh event reading "1h ago" in the admin activity feed. That is fixed; the connection now returns real Date objects, which serialise as ...Z -suffixed UTC and parse correctly in any timezone. If a timestamp is ever off by a suspiciously round number of hours again, a naive date string parsed as local time is the first thing to suspect.

Every table, one line each

Table	Purpose
users	Every buyer, vendor, and admin. See above.
products	Vendor listings. status : active / out_of_stock / removed , all derived rather than set directly.
orders , order_items	Real orders from real checkouts. One vendor per order. order_items.status lets a vendor drop a single line item without cancelling the whole order.
reviews	One per completed order (UNIQUE KEY on order_id), so "verified purchase" is enforced, not a label.

Table	Purpose
reports	The moderation queue: a buyer's complaint against a vendor, or a system auto-flag on a listing containing a restricted term.
report_evidence	A vendor's response and attachments on a report against them. Append-only; it never changes the report's status.
activity_log	The audit trail every admin, vendor, and system mutation writes to. Prunable by a Super Admin, see §5.
vendor_kyc	One row per vendor: CAC number, encrypted identity number, both document URLs, the CheckID.ng provider results, and the review status.
admin_invites	The invite-and-verify flow for adding an admin. A 6-digit code, bcrypt-hashed, 15-minute expiry.
password_reset_tokens	Single-use tokens behind the reset email link, SHA-256 hashed, 1-hour expiry.
two_factor_codes	6-digit sign-in codes for <code>two_factor_enabled</code> accounts. SHA-256 hashed, 10-minute expiry, 5-attempt cap per code, at most one unconsumed row per user.
pending_email_changes	Same shape, for an admin-initiated email change. The OTP goes to the <i>new</i> address; the admin enters it back in the console to finalise. Records which admin requested it.
notifications	Per-user in-app notifications for buyers and vendors: order updates, KYC decisions, account status changes, low stock.
login_ip_history	Every successful login's IP, for admin audit review.
vendor_payout_account_history	Masked account numbers only, one row per link/unlink. The encrypted real number stays on <code>users</code> and is never exposed to admins through any route.
site_banners	The dashboard and explore promo carousel images, admin-managed, picture-only by design.
platform_settings	One row (<code>id</code> always 1), the six Platform Controls toggles on <code>admin/settings.html</code> .
schema_migrations	One row per applied migration filename. Created by the migration runner on first use.

Foreign keys: what cascades and what does not

Most foreign keys here are MySQL's default `RESTRICT`: you cannot delete a row something else still points to. A few are deliberately different, and it matters if you are ever deleting rows by hand.

- `activity_log.actor_user_id` and `admin_invites.invited_by_user_id` are `ON DELETE SET NULL`. Removing a user does not fail just because they have login history or once sent an invite. Those rows survive with the actor reference cleared. This used to be `RESTRICT`, which meant an admin who had ever signed in literally could not be removed from the team; every attempt failed with a generic error.
- `notifications.user_id`, `password_reset_tokens.user_id`, `two_factor_codes.user_id`, and `pending_email_changes` are `ON DELETE CASCADE`. These are per-account ephemera with no value once the account is gone.
- `order_items.order_id` and `report_evidence.report_id` are `ON DELETE CASCADE` onto their parent.
- Everything else referencing `users.id` (orders, products, reviews, reports, KYC, payout history, login history) is still `RESTRICT`, deliberately. A real order or review disappearing because a user row got deleted would be far worse than the delete simply failing.

`DELETE /api/admin/team/:id` works around this properly rather than around the constraint: it runs in a transaction that first nulls that admin's `reports.reporter_user_id`, `reports.attended_by_user_id`, and `vendor_kyc.reviewed_by_user_id` references, then deletes the row, so the records survive with the attribution cleared.

3. Creating a Super Admin without the console

The normal path is an existing Super Admin using `admin/settings.html`'s Add Admin flow: invite, a 6-digit code emailed to the invitee, then a temporary password emailed to them on verification. **This section is for when that is not possible**, because nobody has console access, or the last Super Admin is locked out, or you need one urgently and cannot wait on email delivery.

This still works exactly as written: an admin row needs only `id`, `role`, `name`, `email`, `password_hash`, `admin_role`, and `status`. Everything else on `users` is nullable or has a default (`session_version` defaults to 0, `two_factor_enabled` to false, `last_activity_at` stays `NULL`, which the auth middleware treats as "active now" rather than as an expired session). An admin needs no `state`, `phone`, or `address`.

You need a local `backend/.env` with real `DB_HOST` / `DB_PORT` / `DB_NAME` / `DB_USER` / `DB_PASSWORD` (and `DB_SSL=true` for Clever Cloud) pointed at the live database, plus Node installed and `npm install` already run in `backend/`. From the `backend/` directory:

```
node -e "
require('dotenv').config();
const mysql = require('mysql2/promise');
const bcrypt = require('bcryptjs');
const crypto = require('crypto');

(async () => {
  const pool = await mysql.createPool({
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
    ssl: process.env.DB_SSL === 'true' ? { rejectUnauthorized: false } : undefined,
    connectionLimit: 1,
  });

  const id = crypto.randomUUID(); // a v4 UUID is fine here, see §2 on why v7 matters for high-volume ta
  const email = 'REPLACE_WITH_REAL_EMAIL';
  const name = 'REPLACE_WITH_REAL_NAME';
  const plainPassword = 'REPLACE_WITH_A_REAL_STRONG_PASSWORD';
  const passwordHash = await bcrypt.hash(plainPassword, 10);

  await pool.query(
    `INSERT INTO users (id, role, name, email, password_hash, admin_role, status)
     VALUES (?, 'admin', ?, ?, ?, 'Super Admin', 'active')`,
    [id, name, email, passwordHash]
  );

  console.log('Created Super Admin:', email, '(id:', id + ')');
  await pool.end();
})();
"
```

Replace the three `REPLACE_WITH_*` values first. Then sign in at `https://vetra-vercel.vercel.app/admin/login.html` with that email and password.

Do this carefully:

- The password goes into the command in **plain text**, so it lands in your shell history. Either pick one you will change immediately after first sign-in, or clear it afterwards (`history -d <line>` in bash/zsh), and do not reuse it anywhere else.
- Do not run this against the live database from a machine or network you do not trust. It is a direct, unauthenticated-from-the-app's-perspective write to `users`.
- It bypasses every protection the invite flow provides: no email verification, no proof the person actually received anything. Only do it when you are certain the address belongs to the right person.
- If the goal is to *recover* an existing Super Admin rather than create a new one, prefer resetting that account's password (below) over creating a second Super Admin you then have to remember to remove.

Resetting a locked-out account's password directly

Same connection setup. There is no way to recover a password, only to replace the hash:

```
node -e "
require('dotenv').config();
const mysql = require('mysql2/promise');
const bcrypt = require('bcryptjs');

(async () => {
  const pool = await mysql.createPool({ /* same config as above */ connectionLimit: 1 });
  const hash = await bcrypt.hash('REPLACE_WITH_A_REAL_STRONG_PASSWORD', 10);
  await pool.query(
    'UPDATE users SET password_hash = ?, password_changed_at = NOW(), session_version = session_version + 1 WHERE email = ? AND username = ?'
    [hash, 'someone@example.com', 'admin']
  );
  await pool.end();
})();
"
```

Bumping `session_version` is the part people forget. Without it, any token already issued to that account stays valid, which defeats the point if you are doing this because the account was compromised.

4. Finding users and their details in the database

The admin console does not surface every raw field (payout details, the exact `admin_role` value, `signup_method`, provider KYC messages). For those, query directly. Same connection setup as §3, then plain SQL:

```
-- One user by email. Remember the same address can exist once per role.
SELECT * FROM users WHERE email = 'someone@example.com';

-- Every admin and their role
SELECT id, name, email, admin_role, status FROM users WHERE role = 'admin';

-- Suspended vendors
SELECT id, name, email, store_name FROM users WHERE role = 'vendor' AND status = 'suspended';

-- KYC submissions stuck in manual review
SELECT u.store_name, u.email, vk.status, vk.identity_provider_message, vk.cac_provider_message
FROM vendor_kyc vk JOIN users u ON u.id = vk.vendor_id
WHERE vk.status = 'manual_review';

-- A vendor's payout account, encrypted, see the decrypt note below
SELECT payout_bank_name, payout_bank_code, payout_account_number_enc, payout_account_name
FROM users WHERE id = '...';

-- Recent logins for one account
SELECT ip_address, occurred_at FROM login_ip_history
WHERE user_id = '...' ORDER BY occurred_at DESC LIMIT 20;
```

What you are actually looking at, field by field:

- `password_hash` is a bcrypt hash, starting `$2a$` or `$2b$`. It **cannot be reversed**, which is the entire point. For a locked-out user, the options are the real reset flow (an admin clicking Reset Password, which emails a link) or writing a fresh bcrypt hash as in §3. Never write a plaintext value into this column.
- `payout_account_number_enc` and `vendor_kyc.identity_number_enc` are AES-256-GCM ciphertext in the form `iv:authTag:ciphertext`, all hex. To read either, use the same `decrypt()` the app uses, with the live `ENCRYPTION_KEY`, run from `backend/` with `.env` present:

```
node -e "
require('dotenv').config();
const { decrypt } = require('./src/utils/encryption');
console.log(decrypt('PASTE_THE_ENC_VALUE_HERE'));
"
```

There is deliberately no API route that returns a decrypted payout number. Admin-facing routes only ever show a masked form. The one exception is `GET /api/admin/vendors/:id`, which does decrypt the KYC identity number for the admin reviewing it, since reviewing an identity document against a number you cannot see is not a review.

- Every other column (`name`, `email`, `phone`, `address`, `store_name`, and so on) is plain readable text. Nothing else in `users` is hashed or encrypted.

5. Pruning the activity log

`admin/activity.html` has a "Clear All" button and a per-entry "X". Both are Super Admin only and both are real deletes against `activity_log`.

- **This is irreversible.** There is no soft delete, no trash, nothing to restore from inside the app. Export first if you might want it:
`SELECT * FROM activity_log` through direct database access.
 - **Clearing the log leaves no trace of itself, by design.** Neither route writes a fresh row afterwards. Clearing the log is meant to actually clear it, not leave a new entry behind every time.
-

6. Operational gotchas worth knowing

Connection limits. The database user has its own `max_user_connections` cap, separate from the app's pool size. On Clever Cloud's free plan that cap is 5. `src/db.js` defaults `DB_CONNECTION_LIMIT` to 20, a number chosen back when Namecheap shared hosting was the target, so `render.yaml` pins it to "5" explicitly and the env value wins. Observed real usage has peaked at 3 simultaneous connections; high `Threads_connected` numbers on that server belong to other tenants of the shared instance, not to this app. If a `render.yaml` change does not take effect (Blueprint sync is not always automatic for environment edits), set `DB_CONNECTION_LIMIT` directly in the Render dashboard. If you hit `ER_USER_LIMIT_REACHED` running a one-off script (each opens its own pool, per the examples above), wait a few seconds for old connections to close and pass `connectionLimit: 1` to that script's own `createPool`.

Applying a schema change. Write a new numbered migration file in `backend/migrations/` (plain additive SQL, no down-migration) and let the deploy apply it: Render's start command is `npm run migrate && npm start`, and the runner skips anything already recorded in `schema_migrations`. Do not edit an already-applied file; the runner tracks filenames, not contents, so an edit is silently skipped. Do not apply a change by hand and then retrofit it into `001_init.sql`, which was the old workflow and no longer is. A migration that fails will fail the deploy, which is intentional: better a failed deploy than a live app against a half-migrated schema.

Email failures are silent to whoever triggered them, deliberately. An admin clicking Reset Password always sees "Reset link sent", whether or not delivery succeeded, because surfacing a delivery failure leaks information to someone who may not need it. The real signal is in Render's logs: `[email:failed]` means Resend rejected a real send attempt (reason included) and `[email:not-configured]` means no `RESEND_API_KEY` is set. Every one of those log lines includes the actual code, link, or password as a fallback, so nothing is unrecoverable just because the email did not land. One subtlety already handled in `src/utils/mailer.js`: the Resend SDK does not throw on an API-level failure, it resolves with `{ data: null, error }`, so the error field is checked explicitly. A bare try/catch there silently swallows every failure, which is exactly what happened once with an admin invite.

Rate limits. `src/middleware/rateLimit.js` defines nine independent per-IP limiters: signin 10 per 15 min, admin-signin 5 per 15 min, Google auth 10 per 15 min, signup 20 per hour, password-reset redemption 10 per 15 min, assistant chat 20 per 15 min, 2FA verify 20 per 15 min, 2FA resend 5 per 15 min, and uploads 30 per 15 min. If a legitimate user is locked out, the window is short; there is no admin override, and waiting is the answer.

Real client IPs. These limiters, and `login_ip_history`, depend on `app.set("trust proxy", "loopback, linklocal, uniquelocal")` in `src/app.js`. That preset trusts any number of hops through the standard private ranges and stops at the first public address. It replaced a hardcoded 1, which stopped one hop too early: every row ever written to `login_ip_history` before that fix recorded a private `10.x.x.x` address (Render's own internal routing), and every visitor looked like the same IP to the rate limiters. Historical IP rows from before the fix are therefore useless, not wrong data to investigate.

User-controlled text is escaped at write time, not render time. `activity_log.message`, `notifications.message`, `reports.reason`, and `report_evidence.response_text` are all rendered with `innerHTML` by frontend JS with no escaping of their own. A vendor's store or product name, or a buyer's report reason, could otherwise contain a real `<script>` or `<img onerror=...>` payload that runs in an admin's, vendor's, or buyer's own authenticated session. Every such value is escaped by `src/utils/escapeHtml.js` immediately before the database write, including every `notify()` call site. One related field is escaped at render time instead, deliberately: `admin/assets/ui.js`'s activity feed escapes `actorName` client-side, because that value is never stored, it is a live `JOIN` onto `users.name` computed fresh on each read, so there is no write-time moment to escape it at. If a new route ever writes user-controlled text into one of those four columns, it needs the same write-time treatment. If a new admin view joins in more freeform user data for display, check whether it needs the render-time treatment instead.

KYC documents are not ordinary public URLs. They are uploaded to Cloudinary as `type: authenticated` and referenced by signed URLs, unlike product photos and banners which are plain public CDN links. `POST /api/vendors/me/kyc` validates that submitted document URLs are genuinely Cloudinary authenticated paths, so a vendor cannot point it at an arbitrary host. If KYC document links ever stop loading in the admin console, a Cloudinary credential or cloud-name change is the first thing to check.

The free Render tier sleeps. After roughly 15 minutes without traffic the service spins down, and the next request waits 30 to 60 seconds while it comes back. That is the plan's documented behaviour. A paid plan is the only fix.

There is no self-service "forgot password". Password resets are admin-triggered only: an admin clicks Reset Password on the customer or vendor detail page, the account holder gets the email link, and they set their own new password at `reset-password.html`. Nothing on the sign-in page starts that flow, so a user who is locked out has to contact support.